produced by your environment so you can achieve your daily objectives. Since the environment is a *dynamic process* (a function of time), it may be producing a continuous stream of data of varied size and type. To make things algorithm-friendly, we need to take this environment data and bundle it into discrete packets that we call the *state* (of the environment) and then deliver it to our algorithm at each of its discrete time steps. The state reflects our knowledge of the environment at some particular time, just as a digital camera captures a discrete snapshot of a scene at some time (and produces a consistently formatted image).

To summarize so far, we defined an objective function (minimize costs by optimizing temperature) that is a function of the state (current costs, current temperature data) of the environment (the data center and any related processes). The last part of our model is the RL algorithm itself. This could be *any* parametric algorithm that can learn from data to minimize or maximize some objective function by modifying its parameters. It does *not* need to be a deep learning algorithm; RL is a field of its own, separate from the concerns of any particular learning algorithm.

As we noted before, one of the key differences between RL (or control tasks generally) and ordinary supervised learning is that in a control task the algorithm needs to make decisions and take actions. These actions will have a causal effect on what happens in the future. Taking an action is a keyword in the framework, and it means more or less what you'd expect it to mean. However, every action taken is the result of analyzing the current state of the environment and attempting to make the best decision based on that information.

The last concept in the RL framework is that after each action is taken, the algorithm is given a *reward*. The reward is a (local) signal of how well the learning algorithm is performing at achieving the global objective. The reward can be a positive signal (i.e., doing well, keep it up) or a negative signal (i.e., don't do that) even though we call both situations a "reward."

The reward signal is the only cue the learning algorithm has to go by as it updates itself in hopes of performing better in the next state of the environment. In our data center example, we might grant the algorithm a